NETAJI SUBHASH ENGINEERING COLLEGE

SUBJECT :- Distributed System PEC-IT601 B(6590)

Topic Name :- "**Different types of join operations and their use in DDBMS**"

NAME :- ROHAN KARAK

UNIVERSITY ROLL NO.:- 10900123118

SEMESTER :- 6th **STREAM** :- CSE

CLASS ROLL NO.:- 117 **SECTION :-** B

Introduction & Basic Join Types

1. Introduction :-

In a **Distributed Database Management System (DDBMS)**, data is stored across multiple physical locations connected via a network. Since data is distributed, performing operations such as queries often requires combining tables from different sites.

A **join operation** is used to combine rows from two or more tables based on a related column between them. Join operations are essential in DDBMS because:

- Data is fragmented and distributed.
- Queries often require data from multiple sites.
- Efficient joins reduce communication cost and improve performance.

This report explains major types of join operations and their importance in DDBMS.

---

2. Inner Join :-

Definition:

An **Inner Join** returns only the rows that have matching values in both tables.

Syntax Example:

```
SELECT *
FROM Employee E
INNER JOIN Department D
ON E.Dept_ID = D.Dept_ID;
```

Use in DDBMS:

- Frequently used when related data is stored at different sites.
- Helps combine fragmented relations.
- Optimized using semi-join strategies to reduce data transfer.

Advantage:

- Returns only relevant matched data.
- Reduces unnecessary data retrieval.

---

3. Outer Join :-

Outer Join returns matched rows along with unmatched rows from one or both tables.

Types:

1. Left Outer Join
2. Right Outer Join
3. Full Outer Join

Example (Left Outer Join):

```
SELECT *
FROM Employee E
LEFT JOIN Department D
ON E.Dept_ID = D.Dept_ID;
```

Use in DDBMS:

- Used when missing data must also be displayed.
- Important in distributed systems where some fragments may not contain matching records.

Advantage:

- Preserves incomplete data.
- Useful for reporting and analysis.

Advanced Join Types

4. Equi Join :-

Definition:

An **Equi Join** is a type of join where the joining condition is based on equality (=).

Example:

```
SELECT *
FROM Orders O, Customers C
WHERE O.Customer_ID = C.Customer_ID;
```

Use in DDBMS:

- Most common join condition.
- Efficiently implemented using hash join or merge join.
- Used in horizontal and vertical fragmentation reconstruction.

Advantage:

- Simple and efficient.
- Optimized easily by query processors.

---

5. Natural Join :-

Definition:

A **Natural Join** automatically joins tables based on columns with the same name and compatible data types.

Example:

```
SELECT *
FROM Employee
NATURAL JOIN Department;
```

Use in DDBMS:

- Reduces need to explicitly specify join conditions.
- Used when schemas are standardized across distributed sites.

Advantage:

- Cleaner and shorter queries.
- Avoids duplicate columns in result.

---

6. Semi Join :-

Definition:

A **Semi Join** returns rows from one table where matching rows exist in another table, but does not include columns from the second table.

Example Concept:

Instead of transferring full table data, only required matching rows are sent.

Use in DDBMS:

- Very important for distributed query processing.
- Reduces communication cost between sites.
- Used in distributed join optimization.

Advantage:

- Minimizes data transfer.
- Improves network performance.

Additional Join Types & Importance

7. Self Join :-

Definition:

A **Self Join** joins a table with itself.

Example:

```
SELECT A.Employee_Name, B.Employee_Name AS Manager
FROM Employee A, Employee B
WHERE A.Manager_ID = B.Employee_ID;
```

Use in DDBMS:

- Used for hierarchical data.
- Useful when relationship exists within same distributed fragment.

---

8. Cross Join :-

Definition:

A **Cross Join** returns Cartesian product of two tables.

Example:

```
SELECT *
FROM Product
CROSS JOIN Category;
```

Use in DDBMS:

- Rarely used due to high cost.
- Mostly theoretical or controlled use in distributed queries.

Disadvantage:

- Produces very large results.
- High communication overhead in DDBMS.

Importance of Join Operations in DDBMS

1. Reconstruct fragmented data.
2. Support distributed query execution.
3. Enable data integration from multiple sites.
4. Improve parallel processing.
5. Reduce redundancy through efficient query planning.

Efficient join algorithms like hash join, merge join, and nested loop join are used in distributed environments to optimize performance.

Summary and Observation

Summary

Join operations are fundamental in a Distributed Database Management System. Since data is stored across multiple sites, combining related data requires efficient join processing.

Major types of joins include:

- Inner Join
- Outer Join
- Equi Join
- Natural Join
- Semi Join
- Self Join
- Cross Join

Among these, **Inner Join and Semi Join** are most important in DDBMS because they help reduce unnecessary data transfer and improve query performance.

My Observation

In DDBMS, join operations are more complex compared to centralized databases because:

- Data is physically distributed.
- Network communication cost is high.
- Query optimization is critical.

I observed that **Semi Join plays a very significant role** in distributed query optimization as it minimizes data transfer between sites. Proper selection of join strategy directly impacts system performance, scalability, and efficiency.

Therefore, understanding different join types and their distributed implementation is essential for designing efficient distributed database systems.

B) Write short notes on Fragmentation and it's impact in DDBMS.

=> Introduction :- A Distributed Database Management system (DDBMS) is a database system in which data is stored across multiple physical locations but appears as a single database to users. one of the key techniques used in DDBMS is framentation. Fragmentation helps in distributing data efficiently across different sites to improve performance and reliability.

- What is Fragmentation — Fragmentation is the process of dividing a data base table into smaller pieces called fragments and storing them at different site in distributed system.

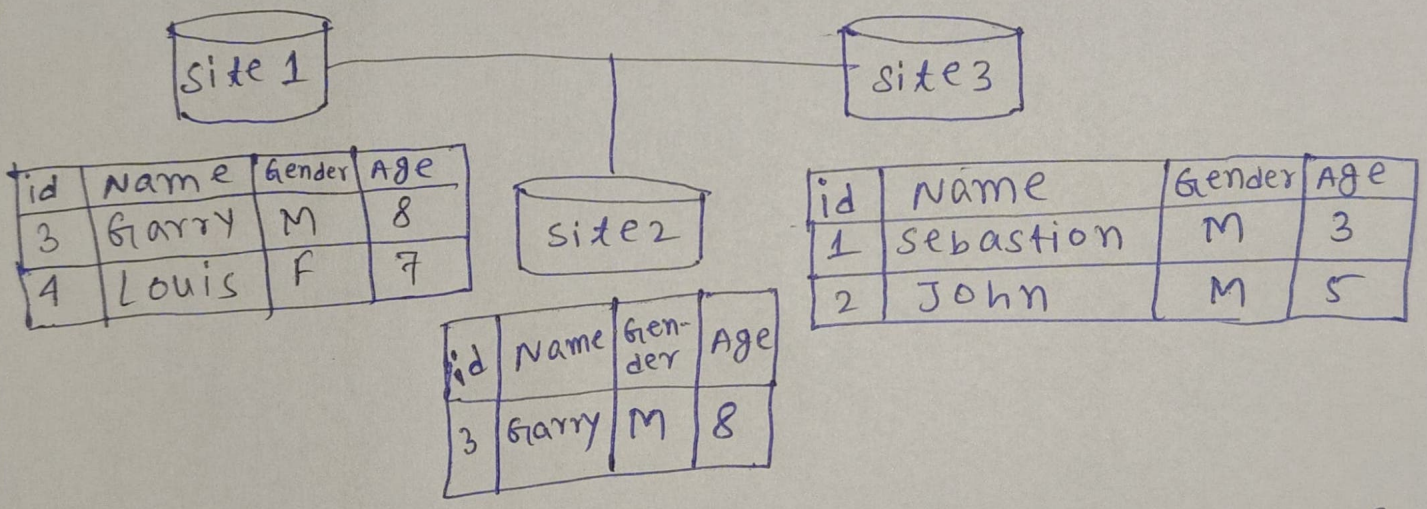
  - The main objective of fragmentation is :-
    - To improve performance.
    - To increase availability
    - To reduce communication cost.
    - To enhance security.

Types of fragmentation:-

1) Horizontal fragmentation:-

Diagram:-

id	Name	Gender	Age
1	sebastion	M	3
2	John	M	5
3	Garry	M	8
4	Louis	F	7



In horizontal fragmentation, the rows (tuples) of a table are divided based on certain conditions.

Example:- A student table can be divided based on location:

- students from kolkata → site 1
- student from Delhi → site 2

Each fragment contains the same attributes but differents rows.

2) Vertical Fragmentation:-

Diagram:-

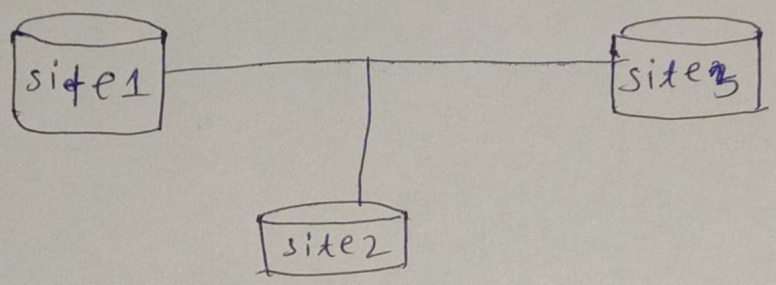
id
1
2
3
4

Name
sebastian
John
Garry
Louis

Gender
M
M
M
F

Age
3
5
8
7

id	Name
1	sebastian
2	John
3	Garry
4	Louis



id	Gender
1	M
2	M
3	M
4	F

id	Name	Gender	Age
1	sebastian	M	3
2	John	M	5
3	Garry	M	8
4	Louis	F	7

In vertical fragmentation, the columns (attributes) of a table are divided.

Ex:- Student table.

- Fragment 1 → (Student_ID, Name)
- Fragment 2 → (Student_Id, marks, Address).

- primary key must be included in each fragment for reconstruction.

3) Mixed (Hybrid) Fragmentation:-

Mixed fragmentation is a combination of Horizontal and vertical fragmentation.

Ex:- First divide by rows (location), then divide colums with in each site.

- Properties of correct fragmentation:-

For proper fragmentation in DDBMS, three rules must be satisfied:

a) Completeness:- All data must be included in fragments.

b) Reconstruction:- original table can be reconstructed from fragments.

c) Disjointness:- Fragments should not overlap (except primary key in vertical fragmentation).

- Impact of Fragmentation in DDBMS:-

- Positive Impacts:-

a) Improved performance:-

- Data is stored near users.
- Faster query processing.

b) Reduced communication cost:-

- Local queries reduce nework traffic.

c) Increased Availability:-

- If one site fails, others can continue working.

Negative Impacts:-

a) complex query processing:-

- Requires Joing fragments from different sites.

b) Data consistency Issues:-

- Difficult to maintain consistency across sites.

c) Higher Design complexity:-

- proper fragmentation strategy is difficult.

conclusion:- fragmentation in DDBMS divides data into smaller parts for better performance and availability. proper design ensures efficiency, while poor increases complexity.